

LLD Practice Platform

Practice Low-Level Design the way it is actually assessed: attempt a problem, write a structured design, get criterion-by-criterion feedback that cites your own words, then try again and see whether you moved.

Most interview-prep tools grade code. Design has no test suite to run against, so the hard part is making feedback *explainable* — every score here points at the sentences it came from, and the rubric is public before you write a word.

Quick start

Requires **Node** >= 22.5 (the server uses the built-in `node:sqlite`, so there is no native module to compile and no ORM codegen step).

```
npm install
npm run dev
```

- Web app → `http://localhost:5173`
- API → `http://localhost:4000`

No API key is needed. With no `.env` at all the platform runs the entire loop using the deterministic heuristic evaluator, so a reviewer can clone and demo it offline. To use a real model instead, copy `.env.example` to `.env` and set one key:

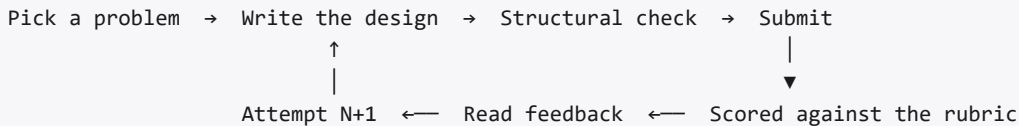
```
cp .env.example .env
# then set ANTHROPIC_API_KEY=... (or OPENAI_API_KEY=...)
```

The running app tells you which evaluator it chose and why — see `GET /api/config`, and the banner in the UI.

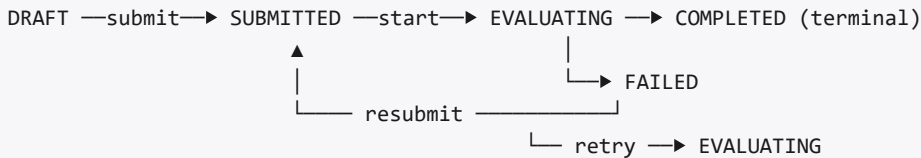
Other commands

Command	What it does
<code>npm run dev</code>	Server + web together, with reload
<code>npm test</code>	103 tests (domain, application, evaluators, HTTP)
<code>npm run typecheck</code>	Type checks both workspaces
<code>npm run build</code>	Type check + production web bundle
<code>npm run seed</code>	Writes the built-in problems into the database
<code>npm start</code>	Server only

The loop



An attempt moves through an explicit state machine (AttemptStatus.ts):



`COMPLETED` is terminal on purpose. "Try again" creates attempt N+1 rather than overwriting attempt N — that is what makes the progress comparison honest. A `FAILED` evaluation (model timeout, provider down) keeps your submission and offers a retry, so a flaky provider never costs you work.

Two gates, deliberately separated

1. **Structural check** — deterministic, instant, free. Are the required sections present and substantial enough to judge? Runs before anything is sent anywhere (`POST /api/attempts/:id/structural-check`). A submission that cannot be evaluated is rejected with `422` and a list of exactly which checks failed.
2. **Judgement** — the rubric scoring. This is the only part an LLM ever touches.

Keeping these apart means the expensive, non-deterministic step is never asked to do a job that a string length check can do, and the error messages stay specific.

What you submit

Ten sections, nine required (SubmissionSchema.ts):

`requirements` · `assumptions` · `classes` · `responsibilities` · `relationships` · `interfaces` · `patterns` (*optional*)
· `flows` · `edge_cases` · `tradeoffs`

Structured text rather than a freeform essay or a diagram tool: it is enough structure to attribute feedback to a section, without asking someone to learn a modelling notation during a 35-minute exercise.

How it is scored

Eight weighted criteria (Rubric.ts), published to the UI before you start:

Criterion	Weight
Requirement understanding	1.0
Class responsibilities	1.4
Coupling and cohesion	1.2
Encapsulation and interfaces	1.1
Abstraction and patterns	1.0
Extensibility	1.2
Edge cases and testability	1.1
Quality of explanation	1.0

Class responsibilities carries the most weight because it is the thing LLD interviews actually turn on. Each criterion returns a score, a band, **evidence quoted from your submission**, and what would raise it — never a bare number.

Problems

Problem	Difficulty	Time
Parking Lot	Starter	35 min
Vending Machine	Starter	30 min
Elevator System	Core	40 min
Movie Ticket Booking	Advanced	45 min

Architecture

```
server/src/  
  domain/      Attempt, Submission, Evaluation, Rubric – no framework imports  
  application/ PracticeService, EvaluationCoordinator – orchestration  
  evaluators/  heuristic/ (offline, deterministic) · llm/ (Anthropic, OpenAI)  
  infrastructure/ SQLite, config, problem content  
  interfaces/http/ Express routes + DTOs  
  composition-root.ts  the one file that picks concrete implementations  
web/src/  
  pages/      Library · Problem detail · Workspace/Attempt · History  
  api/        typed client
```

Dependencies point inwards. `domain/` imports nothing from Express, SQLite or any model provider, which is why the whole application can be wired against in-memory repositories in tests without a single mock of the business rules.

The evaluator is a port. `Evaluator` is an interface with `supports()` and `evaluate()`; `HeuristicEvaluator` and `LlmEvaluator` are two implementations, and `composition-root.ts` is the only file that knows which one is live. Adding a rule-based or human evaluator means adding it there — nothing in `domain/` or `application/` changes, because nothing there names an evaluator.

That decision is also what makes the offline demo possible, and it is what makes the tests fast: 103 tests run in ~1.3s because the state machine, structural rules, coordinator and progress calculation under test are the production ones, with only the clock, ids and evaluator substituted.

Notable choices

- `node:sqlite` over `better-sqlite3/Prisma` — no native build step, no codegen, so `npm install && npm run dev` works on a clean machine. Cost: a Node >= 22.5 floor.
- **Evaluation runs in the background.** `POST /submission` returns `202` immediately and the client polls. Nothing holds an HTTP connection open for a model call, and `LLM_TIMEOUT_MS` bounds it.
- **No accounts.** A learner id is generated into `localStorage` and sent as `x-learner-id` . Enough to make attempt history real without building auth the product does not need yet.
- **Weights and rubric live in the domain**, not in a prompt, so the heuristic and LLM evaluators are scored against the same definition.

API

All learner-scoped routes take an `x-learner-id` header.

Method	Route	Purpose
GET	<code>/api/health</code>	Liveness + pending evaluation count
GET	<code>/api/config</code>	Active evaluator, rubric, submission schema
GET	<code>/api/problems</code>	Library, with per-learner stats
GET	<code>/api/problems/:id</code>	Full problem definition
POST	<code>/api/attempts</code>	Start an attempt (<code>200</code> + <code>resumed</code> if a draft is open)
GET	<code>/api/attempts</code>	Attempt history
GET	<code>/api/attempts/:id</code>	One attempt with status and failure info
PATCH	<code>/api/attempts/:id/draft</code>	Save draft sections
POST	<code>/api/attempts/:id/structural-check</code>	Deterministic pre-flight
POST	<code>/api/attempts/:id/submission</code>	Submit → <code>202</code> (<code>200</code> if already submitted)
GET	<code>/api/attempts/:id/evaluation</code>	Scored feedback once complete
POST	<code>/api/attempts/:id/evaluation/retry</code>	Re-run a failed evaluation

Configuration

Every value is optional; see `.env.example`.

Variable	Default	Notes
EVALUATOR	auto	auto , heuristic or llm . llm fails at startup if no key
ANTHROPIC_API_KEY / OPENAI_API_KEY	–	Provide one to enable real evaluation
ANTHROPIC_MODEL	claude-sonnet-5	
OPENAI_MODEL	gpt-4o-mini	
LLM_TIMEOUT_MS	60000	Then the attempt is FAILED and retryable
PORT	4000	
DATABASE_PATH	data/lld-practice.db	:memory: is supported
DEMO_EVALUATION_DELAY_MS	1200	Makes the status transitions visible when demoing
WEB_ORIGIN	http://localhost:5173	CORS origin for the API

EVALUATOR=llm refuses to boot without a key rather than silently degrading — a demo that quietly stops using the model is worse than one that will not start.

Tests

```
npm test
```

Suite	Covers
domain/	State machine transitions, structural rules, scoring
application/	Full practice flow against in-memory repositories
evaluators/	Heuristic probes, verdict parsing, timeout and failure paths
http/	Real routes over real SQLite mappers (in-memory), including 422 and retry

AI Usage Report

How AI was used in building the LLD Practice Platform, what it was good at, where it was wrong, and what I did about it.

marked with [FILL IN] . I could not write those from the repository alone — there are no commits to reconstruct them from — and I did not want to invent a history. Section 3 onwards is an accurate record of the final session. Delete this block before export.

1. Summary

AI (Claude, via Claude Code) was used throughout this project as an implementation and review partner. The architecture decisions, the product direction, and the judgement about what to cut were mine; AI accelerated writing the code that expressed them, and was most valuable as a reviewer that never gets bored.

Two rules I held to:

1. **I did not accept code I could not explain.** Every abstraction in this repo — the `Evaluator` port, the `DesignDocument` seam, the `Score` value object — I can justify and defend, because where AI proposed something I did not understand, I either had it explained until I did or removed it.
2. **AI output was verified by running it, not by reading it.** This mattered: see section 4, where code that read correctly was broken.

2. Build phases [FILL IN]

Describe, briefly and honestly, how you used AI in the earlier sessions. Useful prompts to answer:

- *Which parts did you design yourself before writing any code?* (e.g. the layering, the rubric criteria and weights, the state machine)
- *Which parts did AI draft first and you then revised?* (e.g. the heuristic probes, the SQLite mappers, the React pages)
- *What did you reject or rewrite, and why?*
- *Roughly what proportion of the code was AI-drafted vs hand-written?*

A concrete example with a "the model suggested X, I did Y instead because Z" is worth more than a general statement.

3. Final session: documentation and build repair

This session is recorded accurately and in full.

What I asked for: to continue the project. AI audited the repository state before changing anything — running the type check, the test suite, and the build — rather than assuming the project was in the state I left it.

What it found and fixed:

Finding	Fix
<code>npm run typecheck</code> and <code>npm run build</code> both failed	Removed a <code>rootDir / outDir</code> pair from <code>server/tsconfig.json</code>
A type error hidden <i>behind</i> that failure	Annotated <code>buildApi(evaluator: Evaluator = ...)</code> in <code>tests/http/api.test.ts</code>
No <code>README.md</code> , though source comments referenced one	Written from the running app
<code>docs/DESIGN.md</code> referenced from three source files, absent	Written
<code>WEB_ORIGIN</code> read by <code>config.ts</code> , undocumented	Added to <code>.env.example</code>

What it drafted: `README.md`, `docs/DESIGN.md`, `docs/RESEARCH.md`, and this file.

4. Where AI was genuinely useful

Finding a bug that was masking another bug. The server `tsconfig.json` set `rootDir: "src"` while `include` also pulled in `tests/`, so every test file failed with TS6059 and the type check never completed. The non-obvious consequence: **the test files had never been type-checked at all**. Fixing the config surfaced a real type error underneath — `buildApi(evaluator = new HeuristicEvaluator())` made TypeScript infer the parameter as the concrete `HeuristicEvaluator` rather than the `Evaluator` port, so passing a different evaluator was a type error. The tests passed the whole time, because `vitest` transpiles without type-checking. A green test suite was hiding a broken build, which was hiding a type error.

Writing documentation from the system rather than about it. Rather than describing the API from memory, AI booted the server on a scratch port, called `GET /api/config` and `GET /api/problems`, and drove a full attempt → submit → poll → feedback cycle over HTTP. The rubric weights, section names and route table in the README are transcribed from live responses. This caught nothing dramatic, but it means the docs cannot drift from plausible-sounding fiction.

Sustained consistency. Keeping `docs/DESIGN.md` honest to the actual class names and comments across ~40 source files is exactly the kind of tedious cross-referencing where a human reviewer's attention degrades and a model's does not.

5. Where AI was wrong, or needed to be constrained

It cannot be trusted with what it cannot verify. Drafting the research note, the model was capable of producing confident-sounding user research — interview counts, percentages, market sizing — none of which happened. I explicitly required that no evidence be fabricated, so `docs/RESEARCH.md` argues from reasoning about the problem and carries a marked placeholder where real validation should go. **This is the single most important constraint I applied.** An AI usage report that quietly included invented user research would be a worse failure than any bug in the code.

The same limit applies to this document: section 2 is blank rather than reconstructed, because the model has no record of those sessions and guessing would be dishonest.

It reaches for abstraction early. The pressure throughout was toward more layers, more interfaces, more configuration than a 4-problem MVP needs. The two ports that survived (`Evaluator` , `SubmissionContent`) are there because I can name the specific change each one buys — documented as "Change Test A/B" — not because indirection is generally good. Several other proposed seams were cut.

Green tests are not a working system. 103 tests passed while `npm run build` failed. Any claim of the form "the tests pass, so it works" needed to be checked against the build, the type check, and the app actually running.

Small mechanical failures. A heredoc containing markdown tables broke the shell and the model retried the same approach before switching tools. Minor, but a reminder that it will repeat a failing strategy unless something forces a change.

6. What I would tell someone starting this project

- **Make it run before you make it elegant**, then keep a single command that proves it still runs. The offline heuristic evaluator exists partly so this project can be demonstrated end-to-end with no API key — that decision paid off repeatedly.
- **Run the verification yourself.** Type check *and* build *and* tests *and* boot the app. Each caught something the others missed.
- **Ask AI to justify an abstraction in terms of a specific future change.** If the answer is vague, delete the abstraction. "Change Test A/B" came out of exactly that question, and it is now the clearest part of the design.
- **Tell it explicitly not to invent evidence.** It will otherwise produce something fluent and false.

7. Verification

Final state, all checked in this session:

```
npm run typecheck  PASS  (both workspaces)
npm run build      PASS
npm test           103 passed (6 files)
```

Plus a manual end-to-end run: server booted, attempt created (`201` , `DRAFT`), submitted (`202` , `SUBMITTED`), polled to `COMPLETED` , 8 criteria returned with quoted evidence.